



貴州財經大學

808
数
据
结
构
考
研
真
题

贵州财经大学 2018 年硕士研究生入学考试

《数据结构》试题 A 卷

一、 名词解释题（共 5 小题，每小题十分，共 50 分）

1. 栈

答：栈(Stack):是限制在表的一端进行插入和删除操作的线性表,又称为后进先出 LIFO (Last In First Out)或先进后出 FILO (First In Last Out)的线性表。

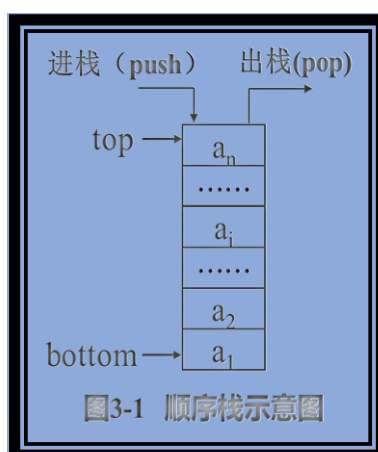
栈顶(Top): 允许进行插入、删除操作的一端，又称为表尾。用栈顶指针(top)来指示栈顶元素。

栈底(Bottom): 是固定端，又称为表头。

空栈：当表中没有元素时称为空栈。

设栈 $S=(a_1, a_2, \dots, a_n)$ ，则 a_1 称为栈底元素， a_n 为栈顶元素，如图 3-1 所示。

栈中元素按 $a_1, a_2, \dots, a_n$ 的次序进栈，退栈的第一个元素应为栈顶元素。即栈的修改是按后进先出的原则进行的。



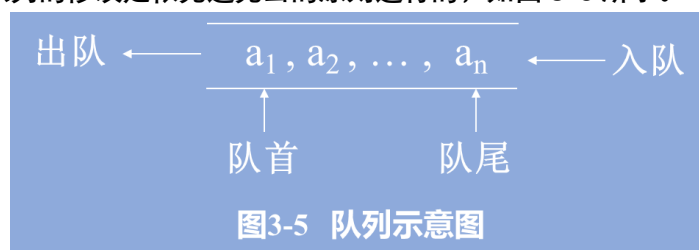
2. 队列

答：队列(Queue):也是运算受限的线性表，是一种先进先出(First In First Out，简称 FIFO)的线性表，只允许在表的一端进行插入，而在另一端进行删除。

队首(front)：允许进行删除的一端称为队首。

队尾(rear)：允许进行插入的一端称为队尾。

空队列：队列中没有元素时称为空队列。在空队列中依次加入元素 $a_1, a_2, \dots, a_n$ 之后， a_1 是队首元素， a_n 是队尾元素。显然退出队列的次序也只能是 $a_1, a_2, \dots, a_n$ ，即队列的修改是依先进先出的原则进行的，如图 3-5 所示。



3. 二叉树

答：二叉树(Binary tree)是 $n(n \geq 0)$ 个结点的有限集合。若 $n=0$ 时称为空树，否则：

(1) 有且只有一个特殊的称为树的根(Root)结点；

(2) 若 $n > 1$ 时,其余的结点被分成为二个互不相交的子集 T_1, T_2 , 分别称之为左、右子树, 并且左、右子树又都是二叉树。

由此可知, 二叉树的定义是递归的。

二叉树在树结构中起着非常重要的作用。因为二叉树结构简单, 存储效率高, 树的操作算法相对简单, 且任何树都很容易转化成二叉树结构。

4. 线性表

答：线性表(Linear List)：是由 $n(n \geq 0)$ 个数据元素(结点) $a_1, a_2, \dots, a_n$ 组成的有限序列。该序列中的所有结点具有相同的数据类型。其中数据元素的个数 n 称为线性表的长度。

当 $n=0$ 时, 称为空表。

当 $n > 0$ 时, 将非空的线性表记作: $(a_1, a_2, \dots, a_n)$, a_1 称为线性表的第一个(首)结点, a_n 称为线性表的最后一个(尾)结点。 $a_1, a_2, \dots, a_{i-1}$ 都是 $a_i (2 \leq i \leq n)$ 的前驱, 其中 a_{i-1} 是 a_i 的直接前驱; $a_{i+1}, a_{i+2}, \dots, a_n$ 都是 $a_i (1 \leq i \leq n-1)$ 的后继, 其中 a_{i+1} 是 a_i 的直接后继。

5. 数据结构

答：数据结构(Data Structure)：是指相互之间具有(存在)一定联系(关系)的数据元素的集合。数据结构由三部分组成：逻辑结构、存储结构(物理结构)、数据的运算。元素之间的相互联系(关系)称为**逻辑结构**。

二、 简答题 (共 2 小题, 每小题 15 分, 共 30 分)

1. 线性结构、树形结构、图状结构(网状结构)中数据元素分别是什么关系?

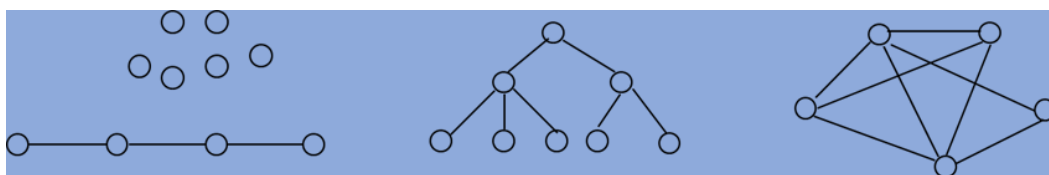
数据元素之间的逻辑结构有四种基本类型, 如图 1-3 所示。

① 集合：结构中的数据元素除了“同属于一个集合”外, 没有其它关系。

② 线性结构：结构中的数据元素之间存在一对一的关系。

③ 树型结构：结构中的数据元素之间存在一对多的关系。

④ 图状结构或网状结构：结构中的数据元素之间存在多对多的关系。



2. 数据的逻辑结构与存储结构有什么区别和联系？

答：数据的逻辑结构与存储结构区别与联系如下：

区别：

逻辑结构：数据元素之间的逻辑关系的描述。常见的逻辑结构由：集合、线性结构、树形结构、图形结构或网状结构 4 种。

存储结构：数据元素在计算机中的存储及其逻辑关系的表现称为数据的存储结构或物理结构。存储结构主要有：顺序存储、链式存储、索引存储和散列存储。

联系：数据的逻辑结构和物理结构是密不可分的两个方面，一个算法的设计取决于所选定的逻辑结构，而算法的实现依赖于所采用的存储结构。

每种逻辑结构采用何种存储结构（物理结构）实现，并没有明确规定，通常根据实现的难易程度，以及在时间和空间方面的要求，选择最适合的物理结构，也不排除复合多种物理结构实现一种逻辑结构的可能。

在 C 语言中，用一维数组表示顺序存储结构；用结构体类型表示链式存储结构。

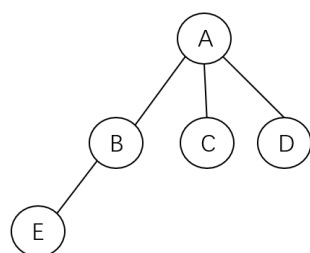
三、 程序设计题(共 1 小题， 每小题 20 分， 共 20 分)

1. 使用递归算法计算 N 的阶乘(使用 C 语言实现)

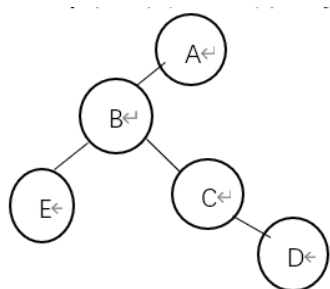
```
#include <stdio.h>
#include <stdlib.h>
int fact(int n)
{
    if( n <= 1)
    {
        return 1;
    }
    return n * fact(n - 1);
}
int main()
{
    int n;
    printf("请输入要计算的阶乘数: ");
    scanf("%d", &n);
    printf("对应结果为: ");
    printf("%d\n", fact(n));
    return 0;
}
```

四、 作图题(共 2 小题， 每小题 10 分， 共 20 分)

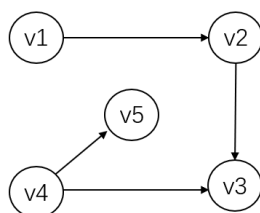
1. 将下面这棵树转换为二叉树。



答： 采用“孩子兄弟表示法”， 转换后的二叉树如下：



2. 写出该图的邻接矩阵。



答： 该图的邻接矩阵如下：

$$\begin{bmatrix} 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

论述题(共 1 小题， 每小题 30 分， 共 30 分)

1、 请论述有哪些排序方法， 分别有什么特点？

答： 排序方法有： P116 页表 7-2 及下面的总结。它们的特点分别如下：

贵州财经大学 2018 年硕士研究生入学考试

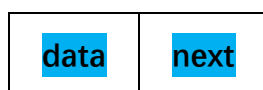
《数据结构》试题 B 卷

一、名词解释题（共 5 小题，每小题十分，共 50 分）

1、链表

答：链式存储：用一组任意的存储单元存储线性表中的数据元素。用这种方法存储的线性表简称线性链表（链表）。存储链表中结点的一组任意的存储单元可以是连续的，也可以是不连续的，甚至是零散分布在内存中的任意位置上的。

data：数据域，存放结点的值。next：指针域，存放结点的直接后继的地址



链表结点结构

2、 广义表

答

:

顾名思义，广义表是线性表的推广，也称为列表。广泛地用于人工智能等领域的表处理语言 LISP 语言，把广义表作为基本的数据结构，就连程序也表示为一系列的广义表。

广义表一般记作

$$LS = (a_1, a_2, \dots, a_n)$$

其中， LS 是广义表 $(a_1, a_2, \dots, a_n)$ 的名称， n 是其长度。在线性表的定义中， a_i ($1 \leq i \leq n$) 只限于是单个元素。而在广义表的定义中， a_i 可以是单个元素，也可以是广义表，分别称为广义表 LS 的原子和子表。习惯上，用大写字母表示广义表的名称，用小写字母表示原子。

显然，广义表的定义是一个递归的定义，因为在描述广义表时又用到了广义表的概念。下面

3、 二叉树前序遍历

答：见 p46

算法的递归定义是：

若二叉树为空，则遍历结束；否则

- (1) 访问根结点；
- (2) 先序遍历左子树(递归调用本算法)；
- (3) 先序遍历右子树(递归调用本算法)。

设 T 是指向二叉树根结点的指针变量，**非递归算法**是：

若二叉树为空，则返回；否则，令 $p=T$ ；

- (1) 访问 p 所指向的结点；
- (2) $q=p \rightarrow Rchild$ ，若 q 不为空，则 q 进栈；
- (3) $p=p \rightarrow Lchild$ ，若 p 不为空，转(1)，否则转(4)；
- (4) 退栈到 p，转(1)，直到栈空为止。

4、**哈希方法**

答： 哈希函数：在记录的关键字与记录的存储地址之间建立的一种对应关系叫哈希函数。

哈希函数是一种映象，是从关键字空间到存储地址空间的一种映象。常用的哈希方法有：

- (1) 直接寻址法
- (2) 数字分析法
- (3) 平方取中法
- (4) 折叠法
- (5 除留取余法；

冲突处理方法：

- (1) **开放寻址法：**
 - ① $d_i=1, 2, 3, \dots, m-1$ ，称线性探测在散列
 - ② $d_i=1^2, -1^2, 2^2, -2^2, 3^2, \dots, \pm k^2$ ($k \leq \lfloor m/2 \rfloor$)，称二次探测在散列
 - ③ d_i =伪随机数序列，称伪随机探测在散列法
- (2) 在散列法
- (3) 链地址法（拉链法）
- (4) 散列表的查找

5、**完全二叉树**

答：完全二叉树(Complete Binary Tree)：如果深度为 k，由 n 个结点的二叉树，当且仅当其每一个结点都与深度为 k 的满二叉树中编号从 1 到 n 的结点一一对应，该二叉树称为完全二叉树。

或深度为 k 的满二叉树中编号从 1 到 n 的前 n 个结点构成了一棵深度为 k 的完全二叉树。

其中 $2^{k-1} \leq n \leq 2^k - 1$ 。完全二叉树的特点：

若完全二叉树的深度为 k ，则所有的叶子结点都出现在第 k 层或 $k-1$ 层。对于任一结点，如果其右子树的最大层次为 i ，则其左子树的最大层次为 i 或 $i+1$ 。

二、简答题（共 2 小题，每小题 15 分，共 30 分）

1、若较频繁地对一个线性表进行插入和删除操作，该线性表宜采用何种存储结构？为什么？

答：宜采用链式存储结构。因为采用链式结构存储线性表，在插入和删除操作时只修改相关节点的指针域，不需要移动节点；而采用顺序结构存储线性表，插入和删除操作需要平均移动表中的一半元素。修改指针域操作比移动元素操作花费的时间少得多。

2、什么是循环队列？

答：为充分利用向量空间，克服上述“**假溢出**”现象的方法是：将为队列分配的向量空间看成一个首尾相接的圆环，并称这种队列为**循环队列**(Circular Queue)。

循环队列为空： $front=rear$ 。

循环队列满： $(rear+1) \% MAX_QUEUE_SIZE = front$ 。

三、程序设计题(共 1 小题，每小题 20 分，共 20 分)

1、对 20 个元素进行升序排列(使用 C 语言实现)

答：#include<stdio.h>

#define N 20

int main(){

int i,j,t,a[N+1];

for(i=1;i<=N;i++)//输入 N 个整数

scanf("%d",&a[i]);

for(i=2;i<=N;i++) //

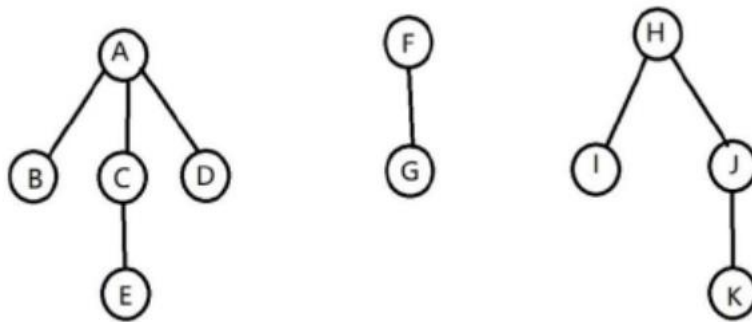
```

for(j=i;j>1;j--){
    if(a[j]>=a[j-1]) break;//按升序排列
    else{t=a[j];a[j]=a[j-1];a[j-1]=t;//如果前面的数大于后面的,交换顺序
    }
}
i=1;
while(i<=N)
    printf("%d ",a[i]);
printf("\n");
return 0;
}

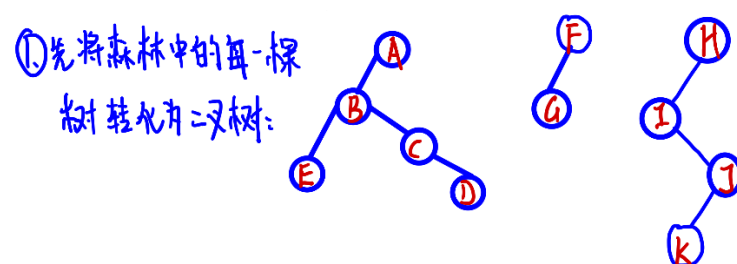
```

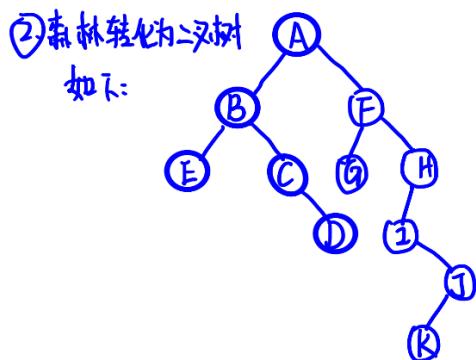
四、 作图题(共 2 小题, 每小题 10 分, 共 20 分)

1. 将下面森林转换为二叉树。

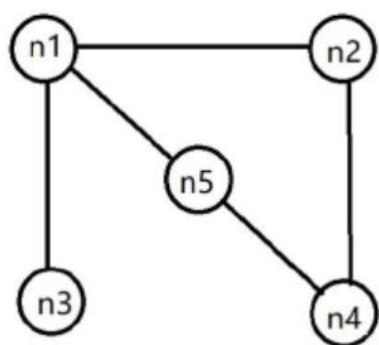


答: 森林转换为二叉树如下:





2、写出该图的邻接矩阵



答：该图的邻接矩阵如下：

$$\begin{bmatrix} 0 & 1 & 1 & 0 & 1 \\ 1 & 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 & 0 \end{bmatrix}$$

五、论述题(共 1 小题， 每小题 30 分， 共 30 分)

1、请论述有哪些查找方法， 分别有什么特点？

答：查找方法有：P90-92 页顺序查找、折半查找、分块查找。它们的特点分别如下：

(答定义+优缺点+适用情况 + 下列的表)

	顺序查找	折半查找	分块查找
ASL	最大	最小	中间

表结构	有序表、无序表	有序表	分块有序
存储结构	顺序表、线性链表	顺序表	顺序表、线性链表

贵州财经大学 2020 年硕士研究生入学考试《数据结构》

一、 名词解释(5x10)

1. 算法

答：**算法(Algorithm)**：是对特定问题求解方法(步骤)的一种描述，是指令的有限序列，其中每一条指令表示一个或多个操作。

算法具有以下五个特性

- ① 有穷性：一个算法必须总是在执行有穷步之后结束，且每一步都在有穷时间内完成。
- ② 确定性：算法中每一条指令必须有确切的含义。不存在二义性。且算法只有一个入口和一个出口。
- ③ 可行性：一个算法是能行的。即算法描述的操作都可以通过已经实现的基本运算执行有限次来实现。
- ④ 输入：一个算法有零个或多个输入，这些输入取自于某个特定的对象集合。
- ⑤ 输出：一个算法有一个或多个输出，这些输出是同输入有着某些特定关系的量。

2. 抽象数据类型

答：**抽象数据类型(Abstract Data Type, 简称 ADT)**：是指一个数学模型以及定义在该模型上的一组操作。

ADT 的定义仅是一组逻辑特性描述，与其在计算机内的表示和实现无关。因此，不论 ADT 的内部结构如何变化，只要其数学特性不变，都不影响其外部

使用。

ADT 的形式化定义是三元组：ADT=(D, S, P)

其中：D 是数据对象，S 是 D 上的关系集，P 是对 D 的基本操作集。

3. 无向完全图

答：**完全无向图**：对于无向图，若图中顶点数为 n ，用 e 表示边的数目，则 $e \in [0, n(n-1)/2]$ 。具有 $n(n-1)/2$ 条边的无向图称为完全无向图。

完全无向图另外的定义是：

对于无向图 $G=(V, E)$ ，若 $\forall v_i, v_j \in V$ ，当 $v_i \neq v_j$ 时，有 $(v_i, v_j) \in E$ ，即图中任意两个不同的顶点间都有一条无向边，这样的无向图称为完全无向图。

完全有向图：对于有向图，若图中顶点数为 n ，用 e 表示弧的数目，则 $e \in [0, n(n-1)]$ 。具有 $n(n-1)$ 条边的有向图称为完全有向图。

完全有向图另外的定义是：对于有向图 $G=(V, E)$ ，若 $\forall v_i, v_j \in V$ ，当 $v_i \neq v_j$ 时，有 $\langle v_i, v_j \rangle \in E \wedge \langle v_j, v_i \rangle \in E$ ，即图中任意两个不同的顶点间都有一条弧，这样的有向图称为完全有向图。

4. 哈夫曼树 (Huffman 树)

答：**最优二叉树(Huffman 树)**：具有 n 个叶子结点(每个结点的权值为 w_i) 的二叉树不止一棵，但在所有的这些二叉树中，必定存在一棵 WPL 值最小的树，称这棵树为 Huffman 树(或称最优树)。其中 WPL 为带权路径长度最短的二叉树。

5. 栈

答：**栈(Stack)**：是限制在表的一端进行插入和删除操作的线性表，又称为后进先出 LIFO (Last In First Out)或先进后出 FILO (First In Last Out)的线性表。

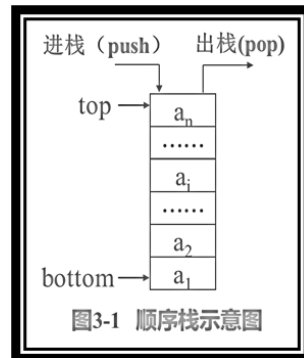
栈顶(Top)：允许进行插入、删除操作的一端，又称为表尾。用栈顶指针(top)来指示栈顶元素。

栈底(Bottom)：是固定端，又称为表头。

空栈：当表中没有元素时称为空栈。

设栈 $S=(a_1, a_2, \dots, a_n)$ ，则 a_1 称为栈底元素， a_n 为栈顶元素，如图 3-1 所示。

栈中元素按 $a_1, a_2, \dots, a_n$ 的次序进栈，退栈的第一个元素应为栈顶元素。即栈的修改是按后进先出的原则进行的。



二、 简答题

1. 最小生成（代价）树 (P76-77)

答：如果连通图是一个带权图，则其生成树中的边也带权，生成树中所有边的权值之和称为生成树的代价。

最小生成树(Minimum Spanning Tree)：带权连通图中代价最小的生成树称为**最小生成树**。构造最小生成树的算法有许多，基本原则是：

- ◆ 尽可能选取权值最小的边，但不能构成回路；
- ◆ 选择 $n-1$ 条边构成最小生成树。

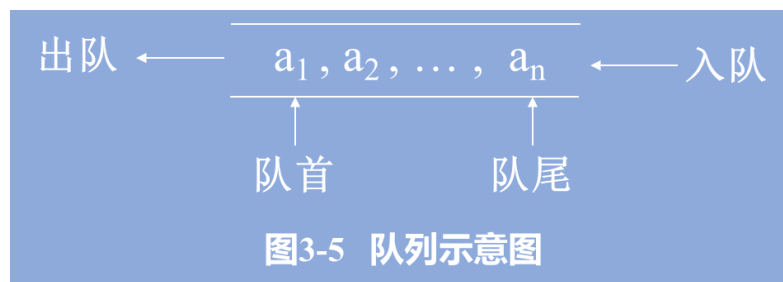
2. 队列

答：队列(Queue):也是运算受限的线性表,是一种先进先出(First In First Out , 简称 FIFO)的线性表, 只允许在表的一端进行插入, 而在另一端进行删除。

队首(front)：允许进行删除的一端称为队首。

队尾(rear)：允许进行插入的一端称为队尾。

空队列：队列中没有元素时称为空队列。在空队列中依次加入元素 $a_1, a_2, \dots, a_n$ 之后, a_1 是队首元素, a_n 是队尾元素。显然退出队列的次序也只能是 $a_1, a_2, \dots, a_n$, 即队列的修改是依先进先出的原则进行的, 如图 3-5 所示。



3. 线性表

答：线性表(Linear List)：是由 $n(n \geq 0)$ 个数据元素(结点) $a_1, a_2, \dots, a_n$ 组成的有限序列。该序列中的所有结点具有相同的数据类型。其中数据元素的个数 n 称为线性表的长度。

当 $n=0$ 时，称为空表。

当 $n>0$ 时，将非空的线性表记作： $(a_1, a_2, \dots, a_n)$ ， a_1 称为线性表的第一个(首)结点， a_n 称为线性表的最后一个(尾)结点。 $a_1, a_2, \dots, a_{i-1}$ 都是 $a_i(2 \leq i \leq n)$ 的前驱，其中 a_{i-1} 是 a_i 的直接前驱； $a_{i+1}, a_{i+2}, \dots, a_n$ 都是 $a_i(1 \leq i \leq n-1)$ 的后继，其中 a_{i+1} 是 a_i 的直接后继。

4. 算法的时间复杂度

答：算法中基本操作重复执行的次数是问题规模 n 的某个函数，其时间度量记作 $T(n)=O(f(n))$ （其中“ O ”是指 $T(n)$ 的数量级），称作算法的渐近时间复杂度(Asymptotic Time complexity)，简称**时间复杂度**。算法的时间复杂度一般用最深层循环内的语句中的原操作的执行频度(重复执行的次数)来表示。常用的时间复杂度的关系为：

$$O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(n^3) < O(2^n) < O(n!) < O(n^n)$$

有的情况下，算法中基本操作重复执行的次数会随问题的输入数据集不同而不同。

三、 算法题

1. 深度优先搜索遍历

答：见 p73-74

2. 随机稀疏矩阵的压缩存储方法

答：见 p36-37

贵州财经大学 2021 年硕士研究生入学考试《数据结构》

一、 名词解释

1. 数据类型

答：数据类型(Data Type)：指的是一个值的集合和定义在该集合上的一组操作的总称。数据类型通常用来（作用）：①约束变量或常量的取值范围；②约束变量或常量的操作。

2. 空间复杂度

答：空间复杂度(Space complexity)：是指算法编写成程序后，在计算机中运行时所需存储空间大小的度量。记作： $S(n)=O(f(n))$ ，其中： n 为问题的规模(或大小)。该存储空间一般包括三个方面：

- ①指令常数变量所占用的存储空间；② 输入数据所占用的存储空间；
- ③辅助(存储)空间。一般地，算法的空间复杂度指的是辅助空间。

一维数组 $a[n]$ ：空间复杂度 $O(n)$

二维数组 $a[n][m]$ ：空间复杂度 $O(n*m)$

3. 广义表

答

:

顾名思义，**广义表**是**线性表**的推广，也称为**列表**。广泛地用于人工智能等领域的表处理语言 LISP 语言，把**广义表**作为基本的数据结构，就连程序也表示为一系列的**广义表**。

广义表一般记作

$$LS = (a_1, a_2, \dots, a_n)$$

其中，**LS**是**广义表** $(a_1, a_2, \dots, a_n)$ 的名称，**n**是其长度。在线性表的定义中， $a_i (1 \leq i \leq n)$ 只限于是单个元素。而在**广义表**的定义中， a_i 可以是单个元素，也可以是**广义表**，分别称为**广义表 LS 的原子和子表**。习惯上，用大写字母表示**广义表**的名称，用小写字母表示**原子**。

显然，广义表的定义是一个递归的定义，因为在描述广义表时又用到了广义表的概念。下面

4. 树

答：树(Tree)是 $n(n \geq 0)$ 个结点的有限集合 T，若 $n=0$ 时称为空树，

否则：

(1) 有且只有一个特殊的称为树的根(Root)结点；

(2) 若 $n > 1$ 时，其余的结点被分为 $m(m > 0)$ 个互不相交的子集 $T_1, T_2, T_3, \dots, T_m$ ，其中每个子集本身又是一棵树，称其为根的子树 (Subtree)。

这是树的递归定义，即用树来定义树，而只有一个结点的树必定仅由根组成。

5. 稀疏图

答：有很少边或弧的图 ($e < n \log n$) 的图称为稀疏图，反之称为稠密图。

二、简答题

1. 定长顺序串

答：**定长顺序存储表示**：将串定义成字符数组，利用串名可以直接访问串值。用这种表示方式，串的存储空间在编译时确定，其大小不能改变。所谓定长顺序存储结构，是直接使用定长的字符数组来定义，数组的上界预先确定。

定长顺序存储结构定义为：

```
#define MAX_STRLEN 256
typedef struct
{ char str[MAX_STRLEN];
  int length;
} StringType;
```

2. 静态链表

答：静态链表借用一维数组来描述线性链表，数组中的一个分量表示一个结点，同时使用游标（指示器 cur 即为伪指针）代替指针以指示结点在数组中的相对位置。数组中的第 0 个分量可以看成头结点，其指针域指向静态链表的第一个结点。这种存储结构仍然需要预先分配一个较大空间，但是在进行线性表的插入和删除操作时不需要移动元素，仅需要修改“指针”，因此仍然具有链式存储结构的主要优点。

3. 线索二叉树

答：见 P51-53

4. 后序遍历

答：见 p48-49

5. 广度优先搜索

答：见 p74-75

三、 算法题

1. 稀疏矩阵的一次定位快速转置算法

答：见 p36-37

贵州财经大学 2022 年硕士研究生入学考试《数据结构》

一、名词解释

1. 算法

答：算法(Algorithm)：是对特定问题求解方法(步骤)的一种描述，是指令的有限序列，其中每一条指令表示一个或多个操作。

算法具有以下五个特性

- ① 有穷性：一个算法必须总是在执行有穷步之后结束，且每一步都在有穷时间内完成。
- ② 确定性：算法中每一条指令必须有确切的含义。不存在二义性。且算法只有一个入口和一个出口。
- ③ 可行性：一个算法是能行的。即算法描述的操作都可以通过已经实现的基本运算执行有限次来实现。
- ④ 输入：一个算法有零个或多个输入，这些输入取自于某个特定的对象集合。
- ⑤ 输出：一个算法有一个或多个输出，这些输出是同输入有着某些特定关系的量。

2. 回路

答：在一条路径中，若没有重复相同的顶点，该路径称为简单路径；第一个顶点和最后一个顶点相同的路径称为回路(环)；在一个回路中，若除第一个与最后一个顶点外，其余顶点不重复出现的回路称为简单回路(简单环)。

3. 森林

答：森林(forest)：是 $m(m \geq 0)$ 棵互不相交的树的集合。显然，若将一棵树的根结点删除，剩余的子树就构成了森林。

4. 链表

答：(线性表) 链式存储：用一组任意的存储单元存储线性表中的数据元素。用这种方法存储的线性表简称线性链表(链表)。存储链表中结点的一组任意的存储单元可以是连续的，也可以是不连续的，甚至是零散分布在内存中的任意位

置上的。

data : 数据域, 存放结点的值。next : 指针域, 存放结点的直接后继的地址

data	next
------	------

链表结点结构

5. 栈

答: **栈(Stack)**: 是限制在表的一端进行插入和删除操作的线性表, 又称为后进先出 LIFO (Last In First Out)或先进后出 FILO (First In Last Out)的线性表。

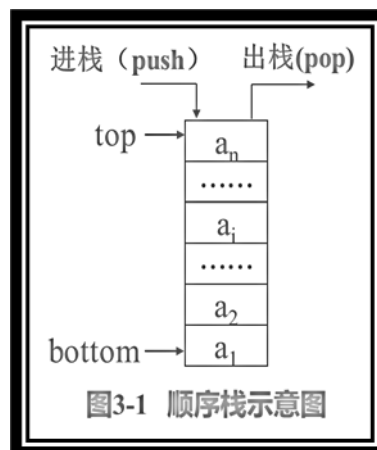
栈顶(Top): 允许进行插入、删除操作的一端, 又称为表尾。用栈顶指针(top)来指示栈顶元素。

栈底(Bottom): 是固定端, 又称为表头。

空栈: 当表中没有元素时称为空栈。

设栈 $S=(a_1, a_2, \dots, a_n)$, 则 a_1 称为栈底元素, a_n 为栈顶元素, 如图 3-1 所示。

栈中元素按 $a_1, a_2, \dots, a_n$ 的次序进栈, 退栈的第一个元素应为栈顶元素。即栈的修改是按后进先出的原则进行的。



二、简答题

1. 二叉树

答: p44-45

2. 双链表

答：见 p19-20

3. 冒泡排序

答：见 p110-111

4. 广度优先搜索

答：见 p74-75

5. 折半查找

答：见 92-93

折半查找又称为二分查找，是一种效率较高的查找方法。

前提条件：查找表中的所有记录是按关键字有序(升序或降序)。

1 查找思想

用 Low、High 和 Mid 表示待查找区间的下界、上界和中间位置指针，初值为 Low=1，High=n。

(1) 取中间位置 Mid: $Mid = \lfloor (Low + High) / 2 \rfloor$;

(2) 比较中间位置记录的关键字与给定的 K 值:

① 相等： 查找成功；

- ② 大于：待查记录在区间的前半段，修改上界指针： $High=Mid-1$ ，转(1)；
- ③ 小于：待查记录在区间的后半段，修改下界指针： $Low=Mid+1$ ，转(1)；
- 直到越界($Low>High$)，查找失败。

```
int Bin_Search(SSTable ST, KeyType key)
{
    int Low=1, High=ST.length, Mid;
    while (Low<High)
    {
        Mid=(Low+High)/2;
        if (EQ(ST.elem[Mid].key, key))
            return(Mid);
        else if (LT(ST.elem[Mid].key, key))
            Low=Mid+1;
        else High=Mid-1;
    }
    return(0); /* 查找失败 */
}
```

三、算法题

1. 括号匹配

答：匹配思想：从左至右扫描一个字符串(或表达式)，则每个右括号将与最近遇到的那个左括号相匹配。则可以在从左至右扫描过程中把所遇到的左括号存放到堆栈中。每当遇到一个右括号时，就将它与栈顶的左括号(如果存在)相匹配，同时从栈顶删除该左括号。

算法思想：设置一个栈，当读到左括号时，左括号进栈。当读到右括号时，则从栈中弹出一个元素，与读到的左括号进行匹配，若匹配成功，继续读入；否则匹配失败，返回 FLASE。

```
#define TRUE 0
#define FLASE -1
SqStack S;
S=Init_Stack(); /*堆栈初始化*/
int Match_Brackets( )
{
    char ch, x;
    scanf("%c", &ch);
    while (asc(ch)!=13)
    {
        if ((ch=='(')||(ch=='['))
            push(S, ch);
```

```

else if ( ch==']' )
{
    x=pop(S);
    if ( x!='[' )
    {
        printf("'[' 括号不匹配");
        return FLASE ;
    }
}
else if ( ch=='(' )
{
    x=pop( S );
    if ( x!='(' )
    {
        printf("'(' 括号不匹配");
        return FLASE ;
    }
}
}
if (S.top!=0)
{
    printf("括号数量不匹配！");
    return FLASE ;
}
else return TRUE ;
}

```

2. 普里姆算法

答：从连通网 $N=(U, E)$ 中找最小生成树 $T=(U, TE)$ 。

算法思想

- (1) 若从顶点 v_0 出发构造， $U=\{v_0\}$ ， $TE=\{\}$ ；
- (2) 先找权值最小的边 (u, v) ，其中 $u \in U$ 且 $v \in V-U$ ，并且子图不构成环，则 $U = U \cup \{v\}$ ， $TE = TE \cup \{(u, v)\}$ ；
- (3) 重复(2)，直到 $U=V$ 为止。则 TE 中必有 $n-1$ 条边， $T=(U, TE)$ 就是最小生成树。

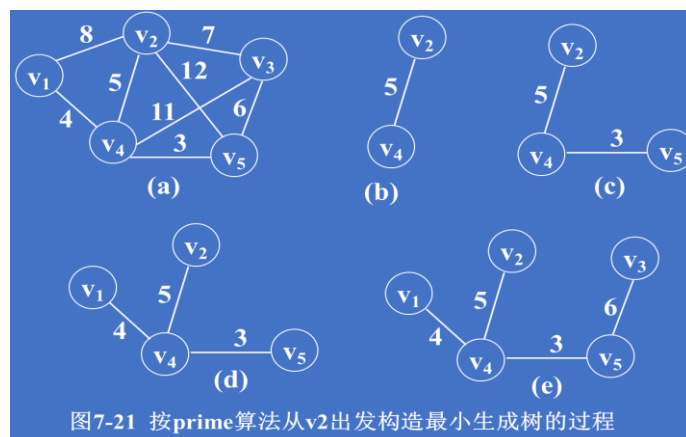


图7-21 按prime算法从v2出发构造最小生成树的过程

算法描述：

```
#define INFINITY MAX_VAL /* 最大值 */
MSTEdge *Prim_MST(AdjGraph *G, int u)
    /* 从第 u 个顶点开始构造图 G 的最小生成树 */
{ MSTEdge TE[]; // 存放最小生成树 n-1 条边的数组指针
    int j, k, v, min;
    for (j=0; j<G->vexnum; j++)
    { closedge[j].adjvex=u ;
        closedge[j].lowcost=G->adj[j][u] ;
    } /* 初始化数组 closedge[n] */
    closedge[u].lowcost=0; /* 初始时置 U={u} */
    TE=(MSTEdge *)malloc((G->vexnum-1)*sizeof(MSTEdge));
    for (j=0; j<G->vexnum-1; j++)
    { min= INFINITY ;
        for (v=0; v<G->vexnum; v++)
            if (closedge[v].lowcost!=0&& closedge[v].Lowcost<min)
            { min=closedge[v].lowcost ; k=v ; }
        TE[j].vex1=closedge[k].adjvex ;
        TE[j].vex2=k ;
        TE[j].weight=closedge[k].lowcost ;
        closedge[k].lowcost=0 ; /* 将顶点 k 并入 U 中 */
        for (v=0; v<G->vexnum; v++)
            if (G->adj[v][k]<closedge[v]. lowcost)
            { closedge[v].lowcost= G->adj[v][k] ;
                closedge[v].adjvex=k ;
            } /* 修改数组 closedge[n]的各个元素的值 */
    }
    return(TE) ;
} /* 求最小生成树的 Prime 算法 */
```

贵州财经大学 2023 年硕士研究生入学考试《数据结构》

一、名词解释：

1. 数据结构
2. 线性结构
3. 链栈
4. 顶点表示活动的网
5. 最坏时间复杂度

二、简答题：

1. 面向对象,
2. 二叉排序树
3. 插入类排序
4. 二叉树还原为森林
5. 先序遍历

三、算法题

1. 判断二叉树的相似性
2. 设计一个算法题求距离顶点 V_0 的最短距离为 K 的算法